

Refinement on Coarse Annotation for Image Segmentation Based YOLO and CRFs

01

Background Introduction

02

**Relationship between the Annotations
& Image Segmentation
& Problem Statement**

03

Objectives

04

**Literature Review &
Solution**

05

Data Preparation

06

Implementation

07

**Conditional Random Fields
(CRF)**

08

Evaluation

09

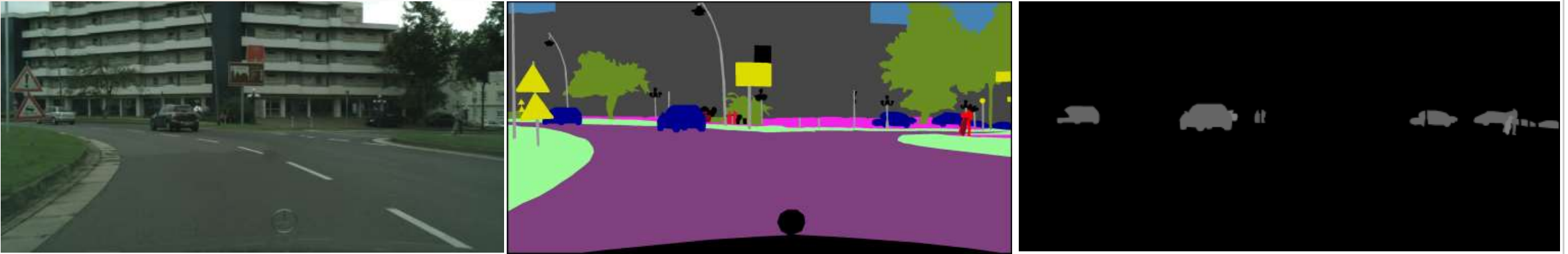
Conclusion & Future Works

01

Background Introduction

Image
Segmentation
and
Annotations

Image Segmentation



Different types of segmentation (a) Raw Image (b) Semantic Segmentation (c) Instance Segmentation

In the field of computer vision, image segmentation is an important task that involves partitioning an image into meaningful segments corresponding to different objects or regions.

The primary goal of image segmentation is to simplify or change the representation of an image into something more meaningful and accessible to analyze.

Annotations

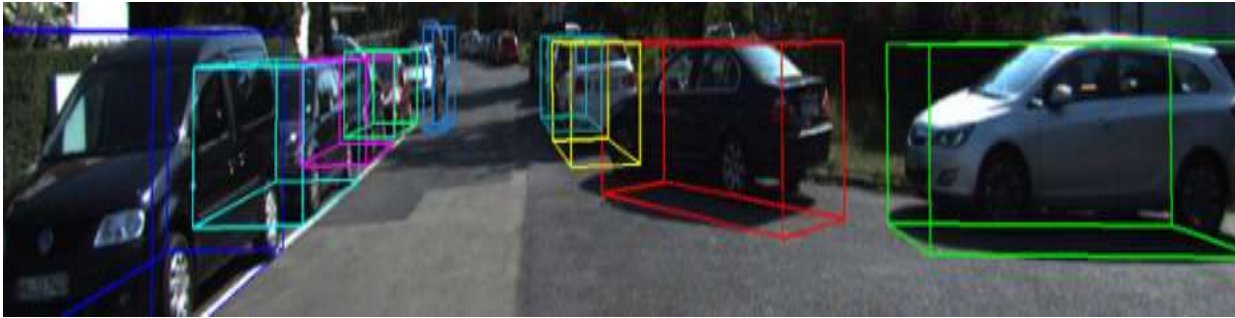


Fine annotations (accurate annotations)

Annotations in computer vision refer to the labelled data used to train and evaluate deep learning models. These labels indicate which parts of an image belong to which classes or objects, often on a pixel-by-pixel basis.

Therefore, in computer vision, accurate and reliable annotations are essential for the adequate performance of models across various applications.

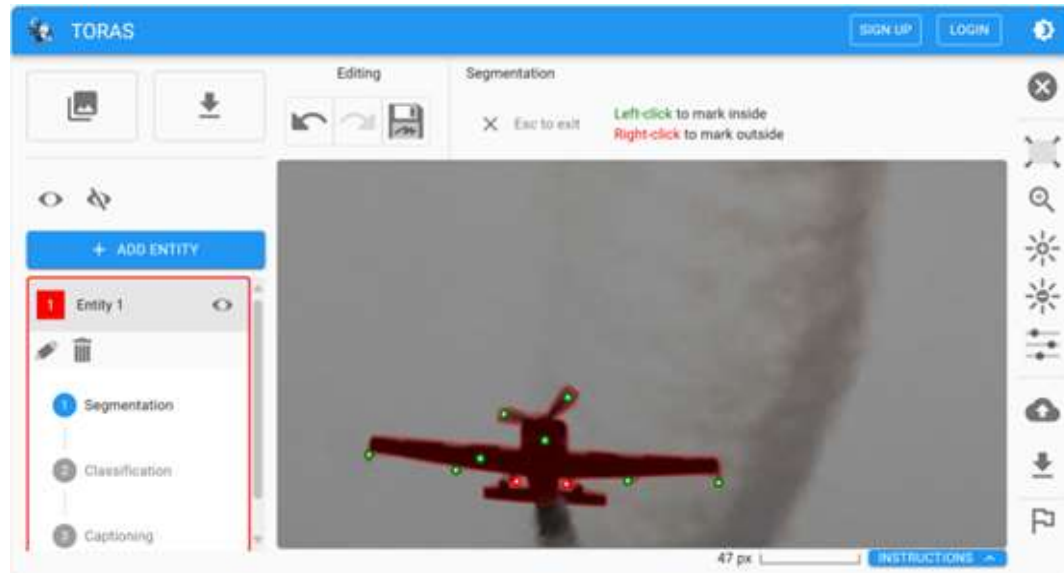
Type of Annotations



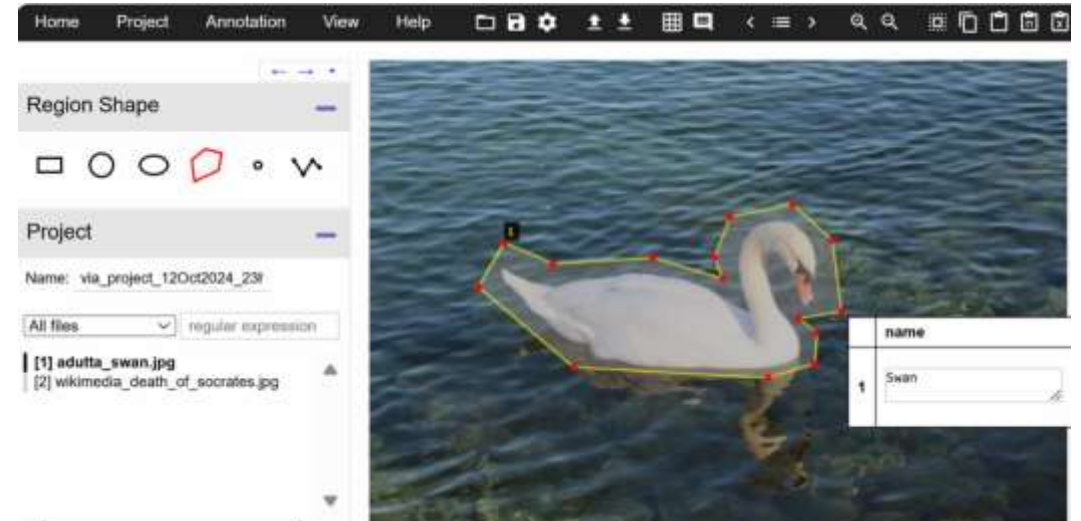
Bounding Boxes



Segmentation Masks



Keypoints



Polygons

02

Relationship between the Annotations & Image Segmentation & Problem Statement

Relationship between the Annotations and Image segmentation

In the training of deep learning models (e.g., U-Net, Mask R-CNN), image segmentation requires a large amount of pixel-level labeled data. Annotations are the direct source of these labeled data, and the degree of detailed annotations directly affects the performance of the segmentation model.

Problem Statement

However, producing these fine annotations is significantly labor-intensive and costly, impractical for large-scale applications. In contrast, coarse annotations are quicker and cheaper to generate but lack the detailed precision necessary for high-accuracy segmentation tasks.

High-quality annotations provide accurate target values that enable the model to effectively capture the true patterns and distributions in the input data, whereas inaccurate annotations cause the model to learn incomplete or incorrect features, which leads to inaccurate prediction results.

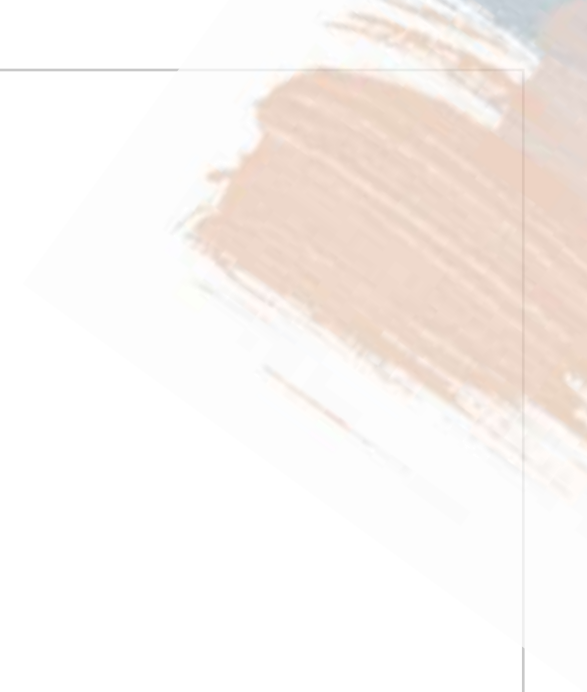
03

Objectives

The purpose of our project is to develop a method that can enhance the quality of coarse annotations to reach the accuracy of fine annotations, thus offering a cost-effective solution for generating high-quality segmentation masks.

04

Literature Review and Solution

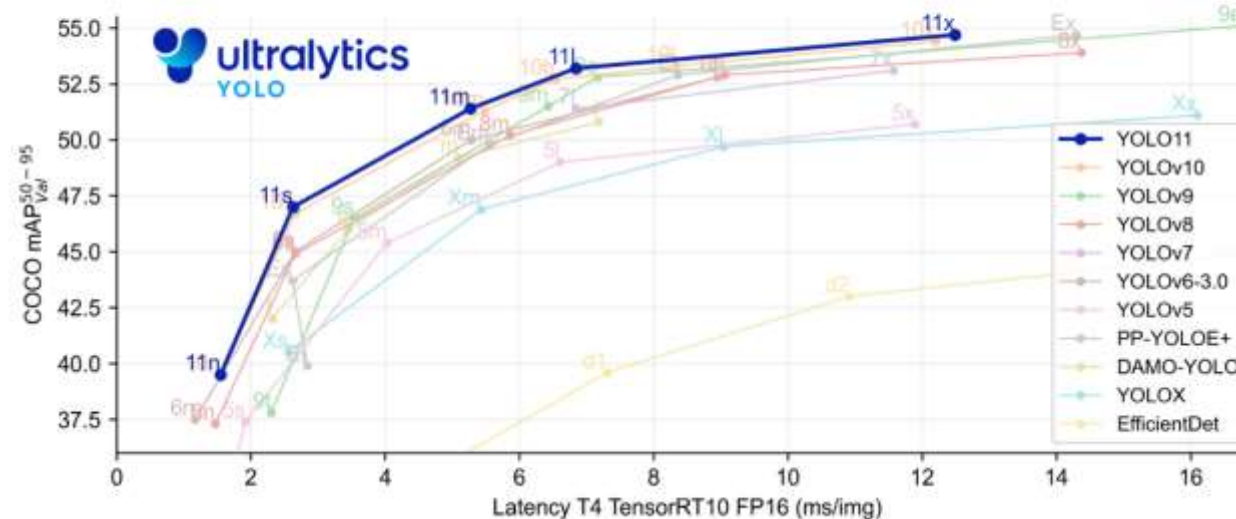


Literature Review: What is YOLOv11?

YOLOv11(You Only Look Once) is the latest object detection model from Ultralytics.

It's about 2% faster in inference time than YOLOv10, while using 22% fewer parameters than YOLOv8m.

YOLO11 is the latest iteration in the Ultralytics YOLO series of real-time object detectors, redefining what's possible with cutting-edge accuracy, speed, and efficiency. Building upon the impressive advancements of previous YOLO versions, YOLO11 introduces significant improvements in architecture and training methods, making it a versatile choice for a wide range of computer vision tasks.



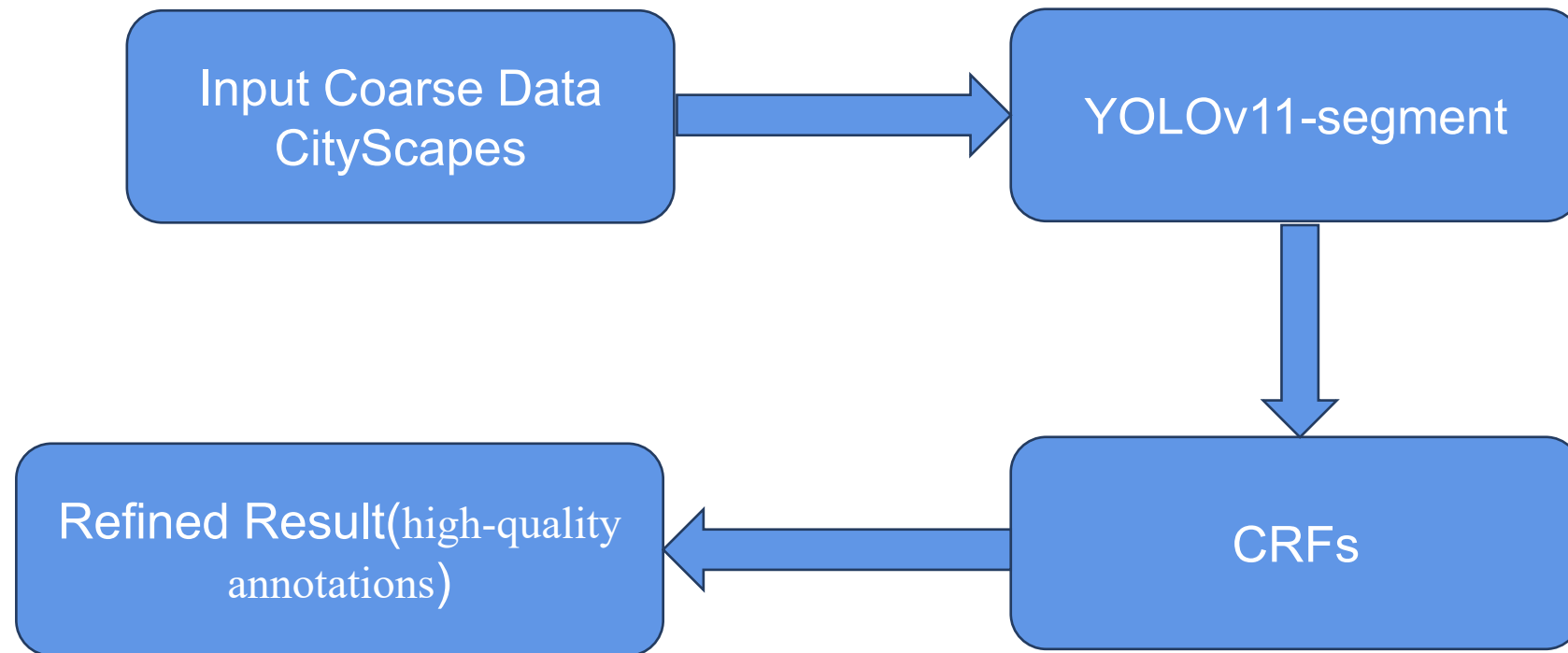
Literature Review: Proposal of Ideas

Initially, the YOLOv11-segment model was pretrained and tested by the COCO (Common Objects in Context) dataset, which is a standard dataset for target detection and segmentation tasks in computer vision.

The predictions generated by YOLOv11 are good, but if we need super-precise, fine-tuned results, Conditional Random Fields (CRFs) is a good choice.

Solution

We propose a framework that combines the YOLOv11-segment model with Conditional Random Fields (CRF) to enhance and refine image annotations on the Cityscapes datasets.

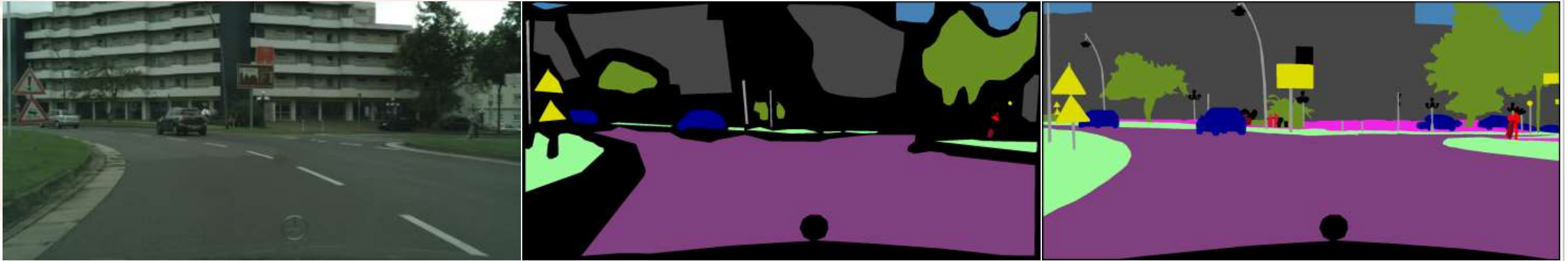


05

Data Preparation:

Datasets Download and Convert





Cityscapes: Coarse annotation and fine annotation

In our project, to better realize the task of semantic segmentation, we choose **Cityscapes** as our dataset, which is specifically tailored for urban scene understanding.

Cityscapes dataset includes 5,000 finely annotated and 20,000 coarsely annotated images, offering a comprehensive resource for training and evaluating segmentation algorithms.

Cityscapes Datasets download

1. We **downloaded three datasets**, gtCoarse.zip, gt_Fine_trainvaltest.zip and leftImg8bit_trainvaltest.zip, from the Cityscapes website.

Among them, gtCoarse.zip is the original roughly labeled dataset, including xxx_gtCoarse_color.png for visualization,

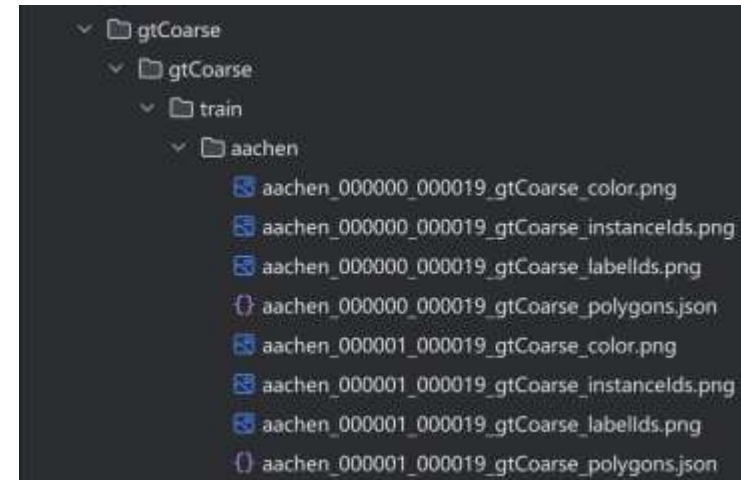
xxx_gtCoarse_instanceIds.png for instance segmentation,

xxx_gtCoarse_labelsIds.png for semantic segmentation,

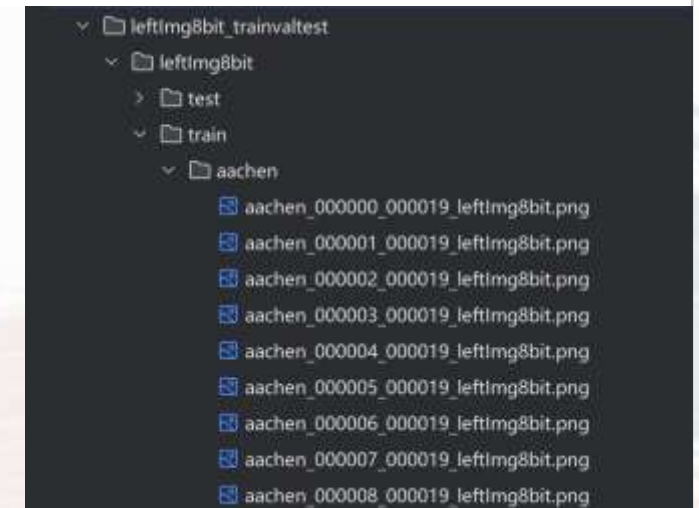
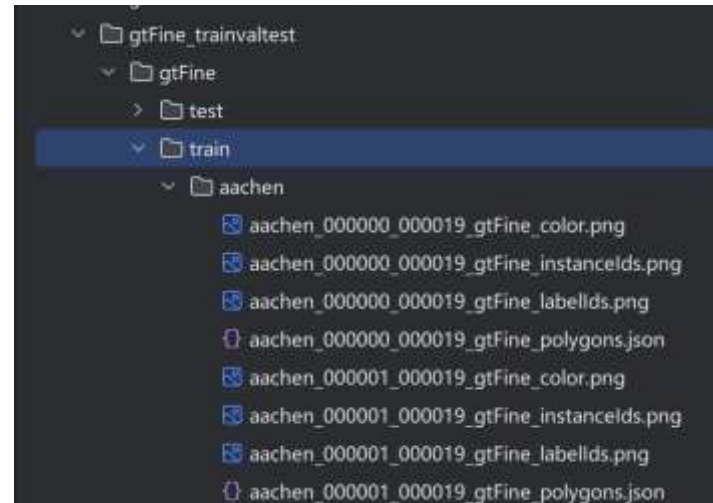
and original human labeling information

xxx_gtCoarse_polygons.json.

2. gt_Fine_trainvaltest.zip is the original fine labeled dataset, the structure is the same as the coarse labeled dataset.



3. leftImg8bit_trainvaltest.zip is the original image file.



Reasons for Data Format Conversion

Due to these three data files in Cityscapes are instance annotations in integer format rather than one hot or probability distribution format. Therefore, the data format needs to be converted.

There are some advantages to converting raw annotated data into one hot format:

- 1. The One hot format ensures that the annotation and prediction formats are completely consistent, making it easy to compare and calculate losses directly.**
- 2. The One hot form clearly expresses the strong constraint that pixels belong to a certain category.**

Conversion of Data Format For gtCoarse and gtFine

generate_one-hot_gtCoarse_instance.py converts xxx_gtCoarse_instanceId.png to a One-hot **.npy file**.

generate_one-hot_gtFine_instance.py converts xxx_gtFine_instanceId.png to a One-hot **.npy file**.

Among them, each pixel in xxx_gtCoarse_instanceIds.png and xxx_gtFine_instanceIds.png is an integer encoded combination, $\text{classId} * 1000 + \text{instanceId}$, Where classId represents the semantic category to which the pixel belongs, InstanceId is the instance identifier within a category, **such as a pixel value of 26001, indicating that the category ID is 26 and the instance ID is 1. However, this type of annotation is hard classification and is not suitable for direct use in model prediction.**

So, convert the value of each pixel into a one hot encoded vector of length num_classes, and save it as a .npy file in NumPy array format. **For example, if num_classes=4, the one hot form of pixels belonging to category 2 is: [0.0, 0.0, 1.0, 0.0]**

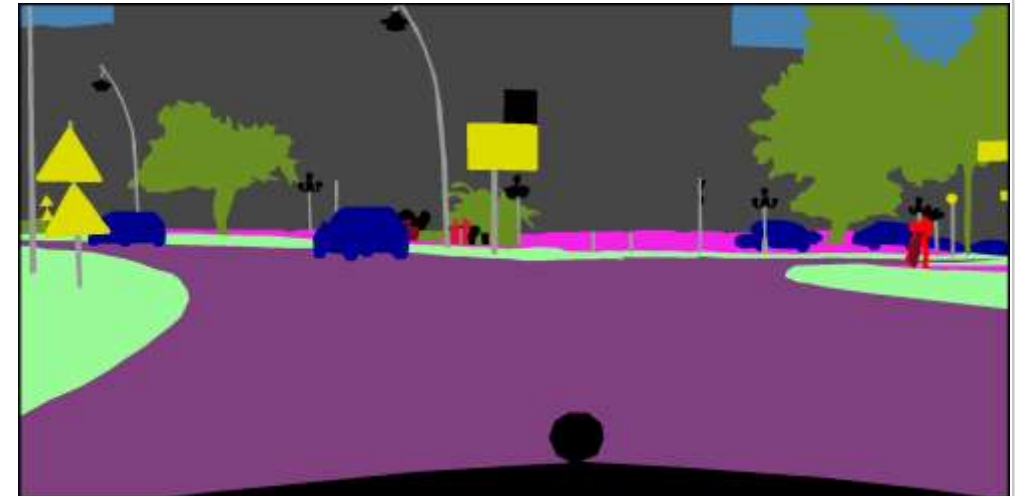
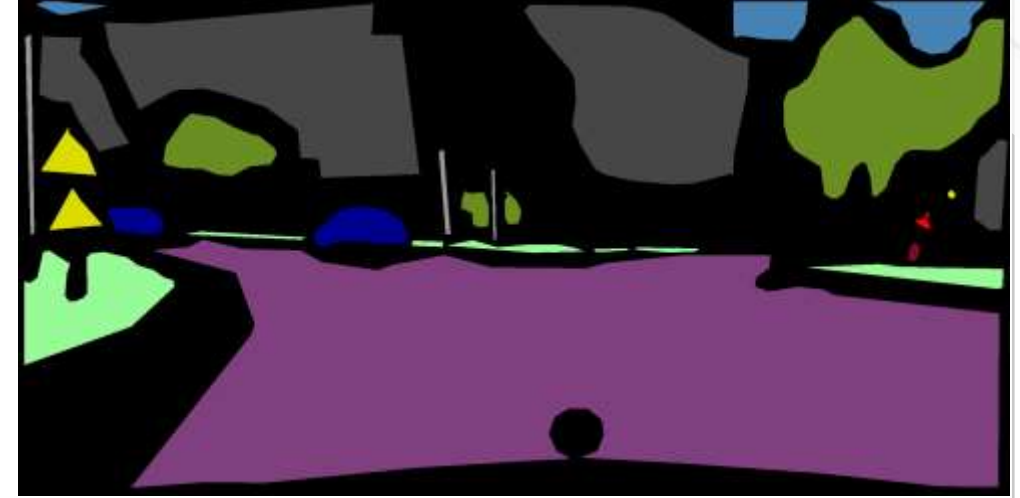
The Converted Result:

1. In the .npy file converted to one-hot format, **since gtCoarse is coarsely labeled, some pixel regions are not assigned category labels, and these pixel regions may be background or ignored labels,** so there are two representations of one-hot format in the converted .npy file.

For unlabeled or ignored pixels the one-hot representation of these pixels is the all-zero vector $[0,0,0,0,0,0]$.

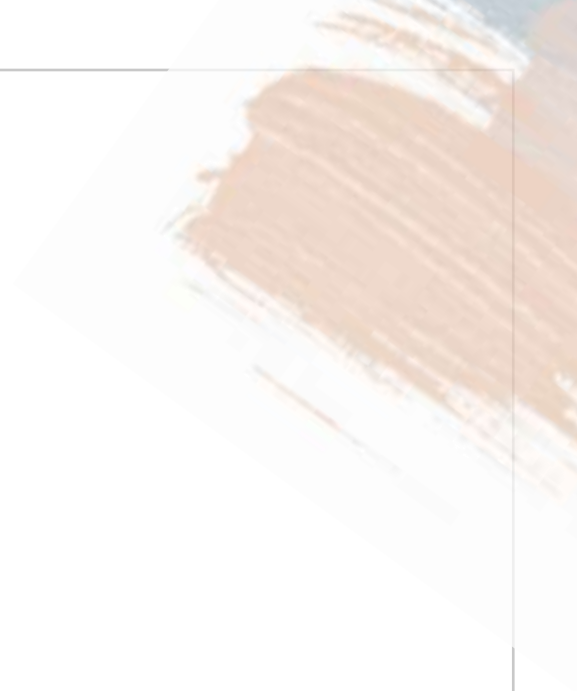
For labeled pixels, the one-hot representation is the normal categorical vector: $[0,0,1,0,0,0]$ (belonging to category 2)

2. However, **since gtFine is finely labeled and all pixels are assigned clear category label distinctions, there are no ignored regions or unlabeled pixels,** so there is always a position 1 in the converted .npy file in one-hot format: $[0,0,0,1,0,0]$ (which belongs to category 3)



06

Implementation



Generate Training Weights

YOLOv11 model comes in 5 variants of increasing size and accuracy: N, S, M, L, and X. At the initial stage of training, we intended to use cityscapes dataset to train YOLO11n-seg.pt and get our own weights best.pt, then we use best.pt to do prediction on Cityscapes coarse dataset, but due to the CPU limitation, the best.pt were not satisfactory.

Models

YOLO11 pretrained Segment models are shown here. Detect, Segment and Pose models are pretrained on the [COCO](#) dataset, while Classify models are pretrained on the [ImageNet](#) dataset.

Models download automatically from the latest Ultralytics [release](#) on first use.

Model	size (pixels)	mAP ^{box} 50-95	mAP ^{mask} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n-seg	640	38.9	32.0	65.9 ± 1.1	1.8 ± 0.0	2.9	10.4
YOLO11s-seg	640	46.6	37.8	117.6 ± 4.9	2.9 ± 0.0	10.1	35.5
YOLO11m-seg	640	51.5	41.5	281.6 ± 1.2	6.3 ± 0.1	22.4	123.3
YOLO11l-seg	640	53.4	42.9	344.2 ± 3.2	7.8 ± 0.2	27.6	142.2
YOLO11x-seg	640	54.7	43.8	664.5 ± 3.2	15.8 ± 0.7	62.1	319.0

- **mAP^{val}** values are for single-model single-scale on [COCO val2017](#) dataset.
Reproduce by `yolo val segment data=coco.yaml device=0`
- **Speed** averaged over COCO val images using an [Amazon EC2 P4d](#) instance.
Reproduce by `yolo val segment data=coco.yaml batch=1 device=0|cpu`

Download yolo11n-seg.pt from the website

```
from ultralytics import YOLO

# Load a pretrained YOLO11 segment model
model = YOLO("yolo11n-seg.pt")

# Train the model
results = model.train(
    #data="CityScapes_Coarse.yaml", # data config file
    data="D:\\Python\\Pycharm\\Projects\\CV_FinalProject_1\\CityScapes_Fine.yaml",
    epochs=15, # training epochs
    imgsz=320, # image size
    batch=4, # number of batches
    optimizer="Adam",
    lr=0.001, # learning rate
    device="cpu", # use CPU
)
```

Generate pre-trained weights best.pt on the Cityscapes

Generate Training Weights

CityScapes
(19 main categories)

VS

COCO
(80 categories)

The distribution of data in the two datasets is somewhat **Overlapping**.

```
from ultralytics import YOLO

# load Fined YOLOv11 model to predict
model = YOLO("yolov11/runs/best.pt")
```

Use our own trained weights to make predictions.

```
from ultralytics import YOLO
import numpy as np
import os
import cv2

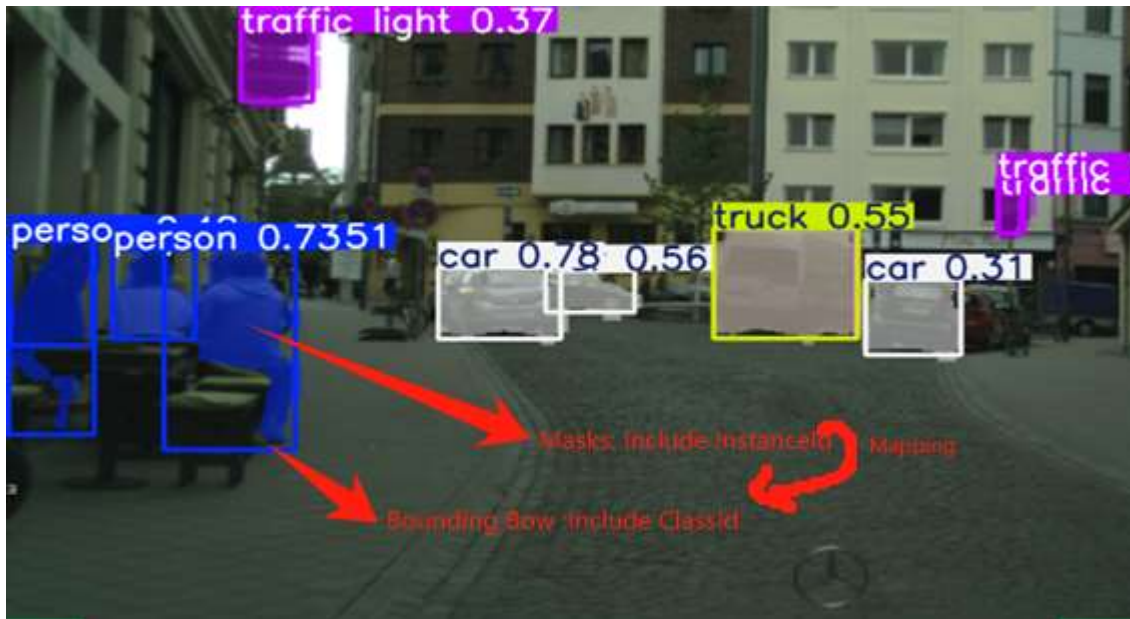
# YOLOv11 model download from https://docs.ultralytics.com/tasks/segment
model = YOLO("yolo11n-seg.pt")

# Image directory
input_dir = "leftImg8bit_trainvaltest/leftImg8bit/train/aachen"
output_dir = "prob_maps_YOLO_instance"
os.makedirs(output_dir, exist_ok=True)
```

Use yolo11n-seg.pt to make predictions.

Prediction on the Original Image

Use `generate_probability_map_YOLO_instance.py` to generate a .npy file form of a `probability_map` on each `xxx_leftImg8bit.png` from the original image file.



Extract Instance Mask and Class from the YOLO Predictions Result.

The Prediction Result

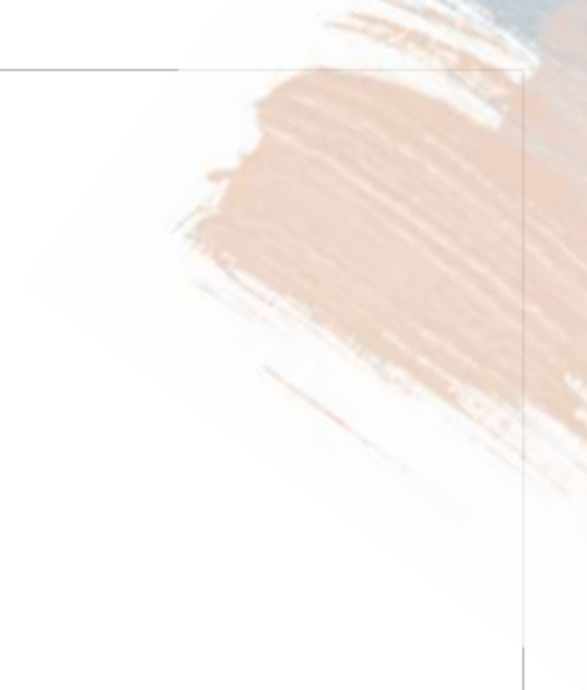
For each pixel (x,y) in the .npy file, its corresponding vector is a probability distribution of length `NUM_CLASSES`. For example, if `NUM_CLASSES = 4`, the probability map vector of a pixel may be `[0.0,0.2,0.7,0.1]`, which means that the pixel has the highest probability of belonging to category 2, which is about 70%.



Visualization of Mapping Result

07

Conditional Random Fields (CRF)



What is CRF?

Conditional Random Fields (CRFs) are probabilistic graphical models used for structured prediction. In image processing, they are commonly employed as a post-processing technique to refine the output of segmentation models.

1. **Unary Potentials:** These represent the likelihood of a pixel belonging to a certain class, derived from the model's predictions.
2. **Pairwise Potentials:** These capture the spatial relationships between pixels, encouraging label consistency within regions and aligning boundaries with image gradients.

Lafferty, John D. et al. "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data." *International Conference on Machine Learning* (2001).

CRF Principle

The refinement process uses CRF to balance the unary potentials from segmentation outputs and pairwise potentials based on pixel proximity and similarity in appearance.

1. **Gaussian Kernel:** Promotes spatial smoothness by considering pixel proximity.
2. **Bilateral Kernel:** Encourages alignment with image edges by considering both spatial proximity and color similarity.

Fully Connected CRFs & Dense CRFs

Fully Connected CRFs extend the traditional CRF by considering every pair of nodes (or pixels) in the graph, not just neighboring ones, making the graph fully connected.

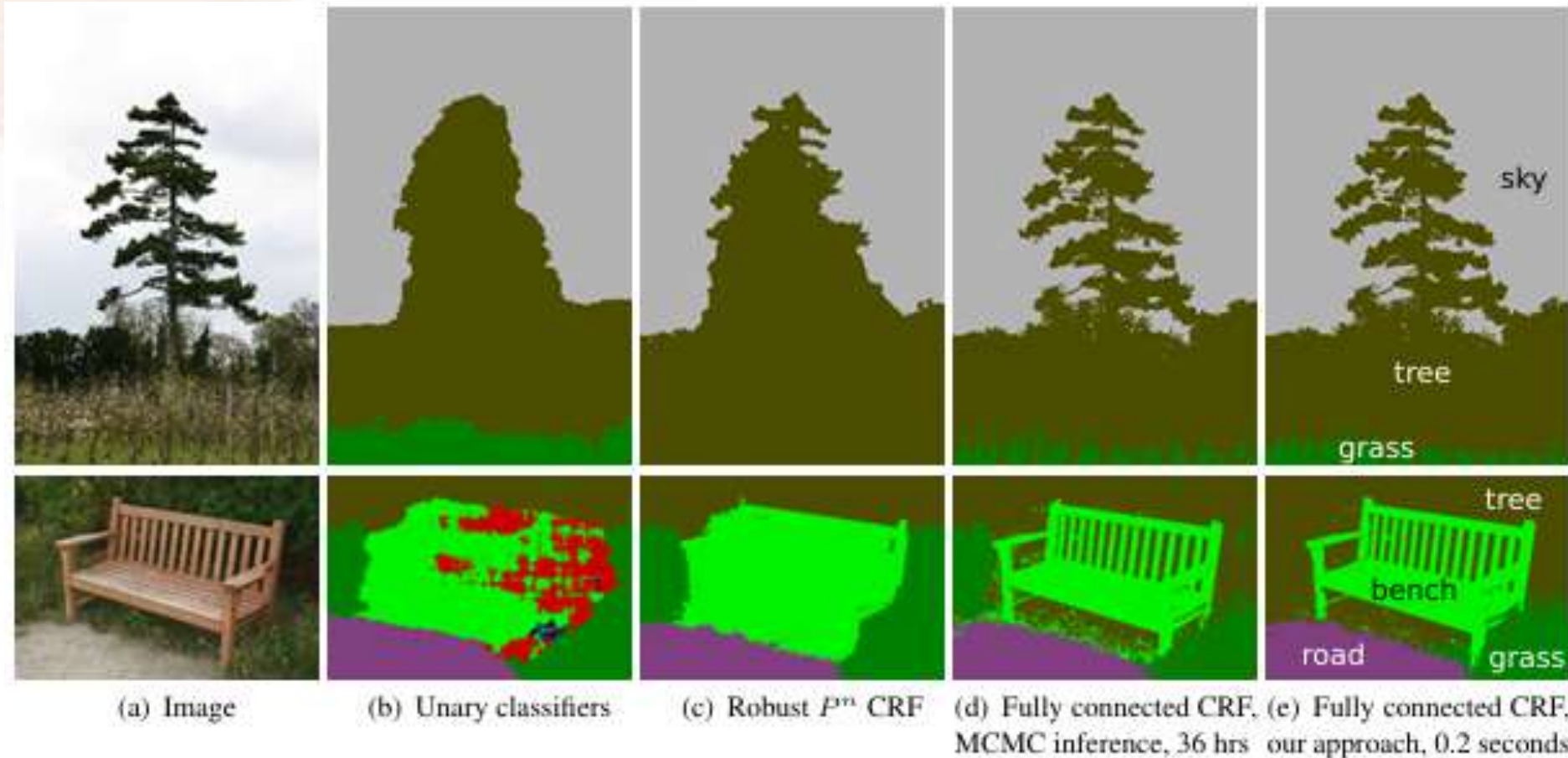
Densecrf use efficient algorithms to handle the large number of connections, making it faster.

Pairwise algorithm:

- Appearance similarity (e.g., color, texture)

- Spatial proximity

Fully Connected CRFs & Dense CRFs



Krähenbühl, Philipp and Vladlen Koltun. "Efficient Inference in Fully Connected CRFs with Gaussian Edge Potentials." (2011)

Pipeline Overview

Input Data

- Coarse annotations.
- Fine annotations for evaluation.
- YOLO model's predictions(probability_map).
- Raw RGB images for bilateral kernel computation.

Processing Steps

- Load probability maps and raw images.
- Apply CRF to refine YOLO and coarse annotations separately.
- Combine the outputs using weighted averaging and intersection.
- Further refine the combined maps using CRF.

Evaluation

- Calculate Intersection over Union (IoU) between the refined maps and ground-truth fine annotations.

Implementation

```
def apply_dense_crf(image, probs, iterations=1, sxy_gaussian=(1, 1), sxy_bilateral=(10, 10), srgb=(8, 8, 8)):
    """
    Refine the probability map using DenseCRF.
    """
    h, w, num_classes = probs.shape
    d = dcrf.DenseCRF2D(w, h, num_classes)

    # Unary potentials
    unary = unary_from_softmax(probs.transpose(2, 0, 1))
    d.setUnaryEnergy(unary)

    # Pairwise potentials (Gaussian kernel)
    gaussian = create_pairwise_gaussian(sxy_gaussian, (w, h))
    d.addPairwiseEnergy(gaussian, compat=3)

    # Pairwise potentials (Bilateral kernel)
    bilateral = create_pairwise_bilateral(sxy_bilateral, srgb, img=image, chdim=2)
    d.addPairwiseEnergy(bilateral, compat=3)

    # Perform inference
    Q = d.inference(iterations)
    refined_probs = np.array(Q).reshape((num_classes, h, w)).transpose(1, 2, 0)
    return refined_probs
```

1. CRF on Individual Annotations:

- DenseCRF is applied separately on the YOLO model's probability maps and the coarse annotations to improve their spatial accuracy.

Combination Strategies

```
def combine_annotations(yolo_probs,  
coarse_probs, weight_yolo=0.9,  
weight_coarse=0.1):
```

```
    """
```

```
        Combine YOLO probabilities and  
coarse annotations using weighted  
averaging.
```

```
    """
```

```
        combined_probs = weight_yolo *  
yolo_probs + weight_coarse *  
coarse_probs  
        return combined_probs
```

2. Weighted Averaging:

Combines YOLO and coarse probabilities by assigning weights to each source.

Sequential Combination

Approach 1
Use CRF on YOLO

```
best_prob_0 = combine_annotations(yolo_prob, coarse_prob)
best_prob_1 = combine_annotations(best_prob_0,
coarse_crf_probs)
```

Approach 2
Use CRF on Coarse

```
final_mask = np.argmax(best_prob_1, axis=-1)
```

Approach 3
Use CRF on combine(coarse + YOLO)

Approach 4
Combine(CRF(YOLO) + Coarse)

Approach 5
Combine(CRF(Coarse) + YOLO)

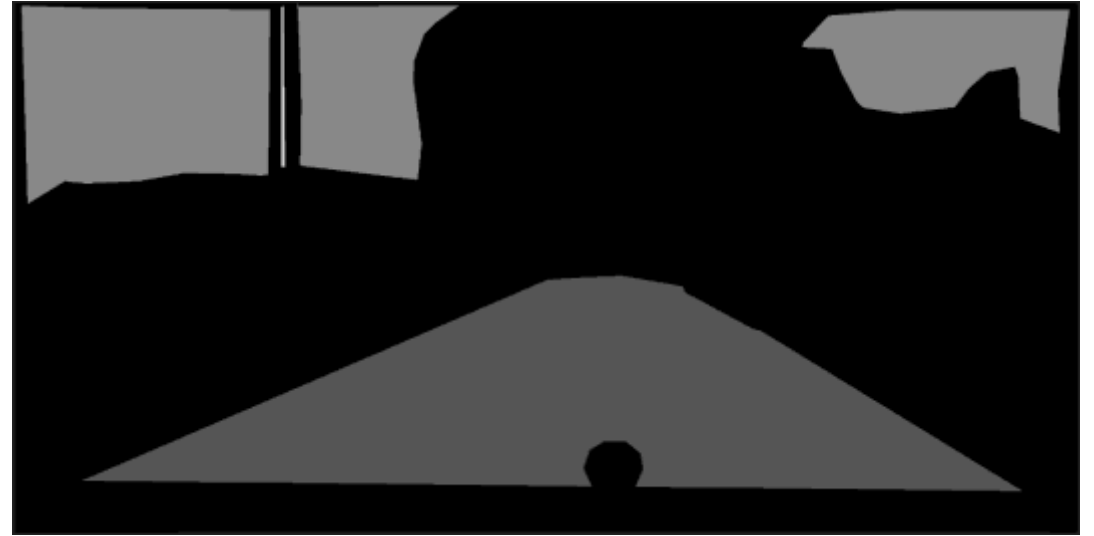
**Combines weighted annotations,
refines them with CRF, and then
merges them with coarse-CRF
outputs to create a refined mask.**

CRF Tuning

Input Coarse Annotations (Most zeros)

Tuning goal:

1. Enhance edge information in non-zero areas.
2. Minimize global smoothing to prevent altering the overall distribution significantly.



CRF Tuning Parameters

Compatibility Term

Function: Balances the weight between different energy terms (e.g., unary and pairwise), influencing the strength of the smoothing effect.

Spatial Gaussian Kernel Range

Function: Controls the spatial range for global smoothing.

Spatial Bilateral Kernel Range

Function: Enhances connectivity in regions with similar colors and spatial proximity.

RGB Similarity in Bilateral Kernel

Function: Controls the influence of color similarity on boundary refinement.

Iterations: Determines the number of iterations during CRF inference.

CRF Tuning



Size: (1024,2048)

Compatibility Term

Increase

Spatial Gaussian Kernel Range

reduce to (1,1) or (2,2)

Spatial Bilateral Kernel Range

Reduce to (20,20)

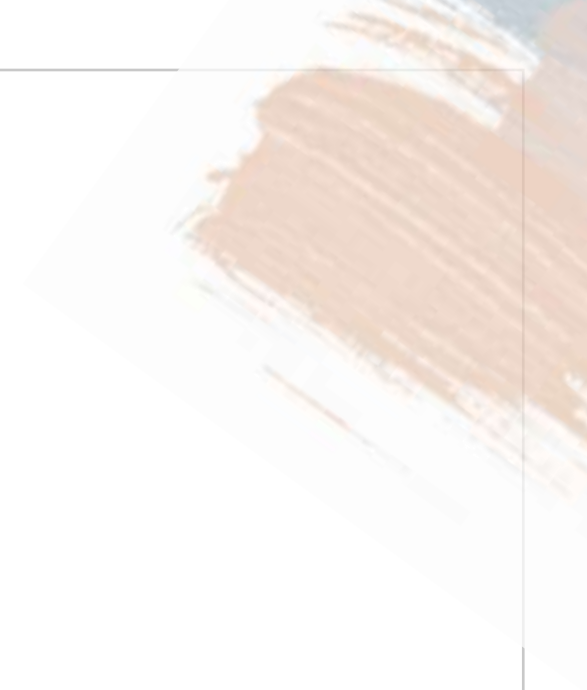
RGB Similarity in Bilateral Kernel

Reduce to (10,10)

Iterations: proper iteration

08

Evaluation



Evaluation with IoU

```
def calculate_iou(mask1, mask2):  
    """  
    Calculate the Intersection over Union  
(IoU) between two binary masks.  
    """  
    intersection = np.logical_and(mask1,  
mask2).sum()  
    union = np.logical_or(mask1,  
mask2).sum()  
    if union == 0:  
        return 0.0  
    return intersection / union
```

1. **IoU Definition:** IoU measures the overlap between the predicted mask and the ground truth mask, defined as:

$$\text{IoU} = \frac{\text{Intersection}}{\text{Union}}$$

generate pre-training weights best pr

Evaluation with IoU

Approach 1
Use CRF on YOLO

Approach 2
Use CRF on Coarse

Approach 3
Use CRF on combine(coarse + YOLO)

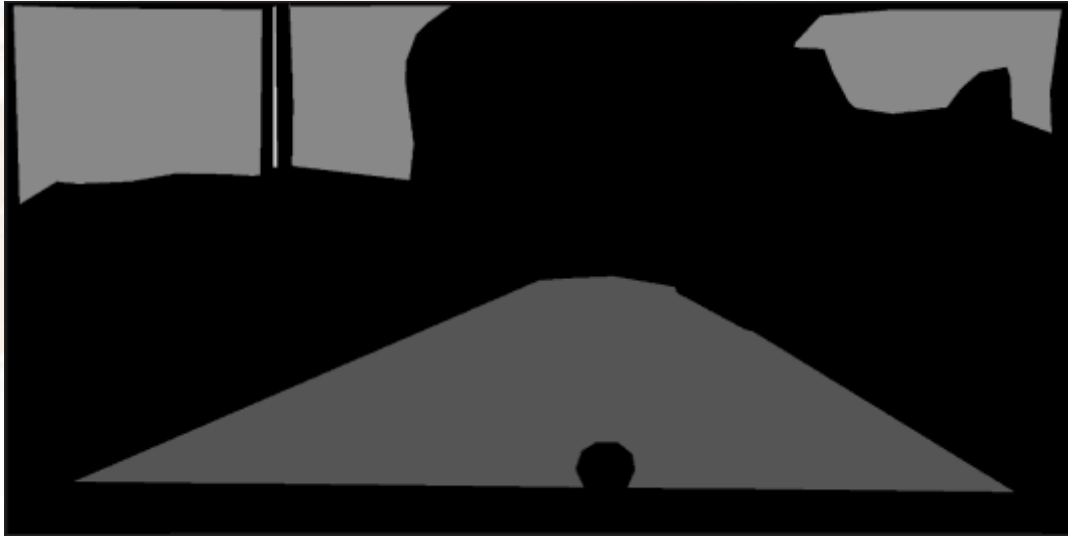
Approach 4
Combine(CRF(YOLO) + Coarse)

Approach 5
Combine(CRF(Coarse) + YOLO)

1. **IoU Definition:** IoU measures the overlap between the predicted mask and the ground truth mask, defined as:

$$\text{IoU} = \frac{\text{Intersection}}{\text{Union}}$$

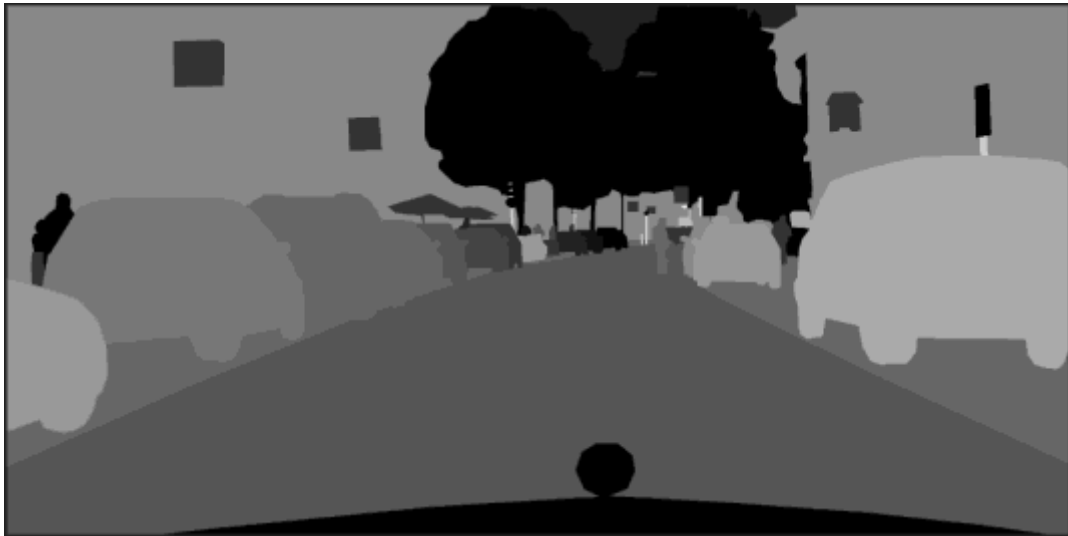
generate pre-training weights best pr



Coarse



Best: combine (CRF (Coarse) + YOLO)



Fine

```
Processing: aachen_000049_000019  
IoU (Coarse vs Fine): 0.3933  
IoU (Best vs Fine): 0.6655
```

Example: aachen_000049_000019

Evaluation on aachen subset

Aachen is one of the cities included in the Cityscapes dataset

Due to memory and limited budget, we only evaluate the **mean IoU(mIoU)** on the first 50 images of this subset.

Table 3: With the fixed budget, quantitative results of FCN-8s and annotation time for semantic segmentation on Cityscapes validation set.

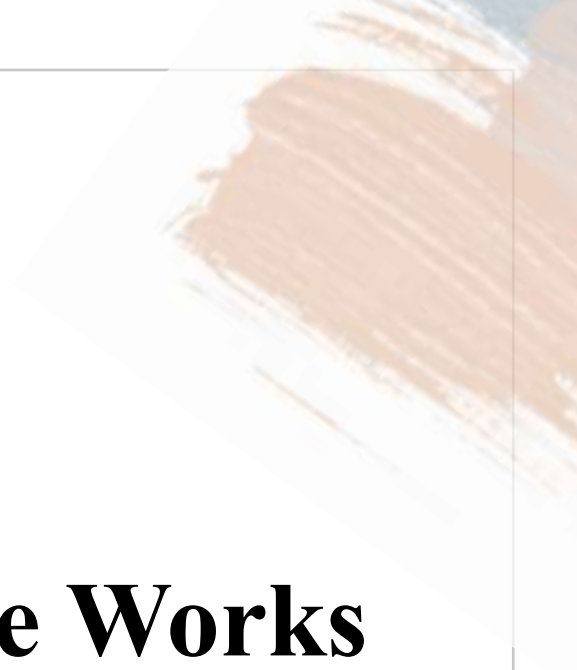
Supervision (fixed budget)	time(h)	mIoU(%)	Acc.(%)
Fine(389 img)	583	32.5	57.8
Coarse(5k img)	583	28.1	48.8
CRF+Coarse(5k img)	583	47.2	64.6
Filling(0.7)+Coar.(5k img)	583	54.7	69.1
Filling(0.3)+Coar.(5k img)	583	55.2	72.6

“Coarse-to-Fine Annotation Enrichment for Semantic Segmentation Learning.” ACM(2018)

For 50 images, the refined average IOU is 0.6564
For 50 images, the coarse average IOU is 0.5664

09

Conclusion & Future Works



Conclusion

We proposed a method to refine coarse annotations using YOLO and CRFs enhances the quality of semantic segmentation, bridging the gap between coarse and fine annotations in datasets like Cityscapes.

The implementation demonstrates that combining multiple refinement strategies can achieve superior results. CRFs remain a powerful tool for post-processing in computer vision tasks, particularly when paired with innovative combination techniques.

Future Works

Class Mapping & Ontology

The YOLO predictions are saved in a probability map consisting of a series of classes. However, these predicted classes may mismatch with those in the original coarse dataset. Developing methods to evaluate the accuracy of class mapping is essential. This involves creating an ontology to understand and align mismatched classes.

Evaluation Methods

Incorporate class-based evaluation metrics such as class-wise mean precision and recall. These methods allow a more granular analysis of individual class performance, ensuring robust evaluation of newly generated classes.

Active Learning Integration

Use machine learning models to generate low-confidence probability maps for manual refinement. Active learning can prioritize these low-confidence regions for human annotation, optimizing labeling efforts and improving dataset quality.

Future Works

Scalability

YOLO models can predict more classes than those labeled in the dataset. This capability opens the possibility of expanding the number of classes in the ground truth annotations, for instance, increasing the Cityscapes classes (34) to align with COCO's 80 classes.

Our method is able to do dataset augmentation with richer class diversity, thereby improving segmentation models. Here we just set 20 classes for model prediction which can be further scalable.

Model Improvement:

Transition from YOLO-based instance segmentation to models tailored for semantic segmentation, such as Mask R-CNN and Deeplab. These models are more suitable for dense segmentation tasks and can yield improved results and enhance model's robustness.

References

Chen, L.-C., Papandreou, G., Kokkinos, I., Murphy, K., and Yuille, A. L. (2023). Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs. IEEE transactions on pattern analysis and machine intelligence, xx:xx–xx.

Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., and Schiele, B. (2016). The cityscapes dataset for semantic urban scene understanding. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 3213–3223.

Everingham, M., Van Gool, L., Williams, C. K., Winn, J., and Zisserman, A. (2023). The pascal visual object classes (voc) challenge: A retrospective. International Journal of Computer Vision, 88(2):303–338.

Krähenbühl, Philipp and Vladlen Koltun. (2011) “Efficient Inference in Fully Connected CRFs with Gaussian Edge Potentials.”

Lafferty, J., McCallum, A., and Pereira, F. C. (2001). Conditional random fields: Probabilistic models for segmenting and labeling sequence data. ICML.

References

Li, Y., Qi, H., Dai, B., Li, H., and Lin, D. (2023). Fully convolutional instance-aware semantic segmentation. *IEEE Transactions on Image Processing*.

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., and Zitnick, C. L. (2014). Microsoft coco: Common objects in context. *European conference on computer vision*, pages 740–755.

Liu, Z., Wang, J., Li, J., Liu, P., and Ren, K. (2023c). A novel improved mask rcnn for multiple targets detection in the indoor complex scenes. *arXiv preprint arXiv:2302.05293*.

Luo, Yadan et al. “Coarse-to-Fine Annotation Enrichment for Semantic Segmentation Learning.” *Proceedings of the 27th ACM International Conference on Information and Knowledge Management* (2018): n. pag.

Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. (2016). You only look once: Unified, real-time object detection. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788.

Ultralytics (2024). Yolov11: Pioneering breakthroughs in object detection technology. *Ultralytics Documentation*.